

Sprint Documentation

Sprint 16	Start Date: 30/04/2026	Final Date: 02/05/2026
Project Name:	civitas-backend	
Ano: 5º	Análise e Desenvolvimento de Sistemas - AMS	

Team Members

Wendell Alves Nascimento
João Antonio Gomes Cestonaro

Sprint Backlog

Task #	Description	Start Date	Final date
#92	Correção e adição de atributos em Despesa	30/04/2026	02/05/2026

Task Description

Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
#92	Modificar a classe Despesa no backend, removendo campos que não serão mais utilizados diretamente e adicionando os novos atributos necessários para o controle de valores previstos, valores pagos, consumo previsto, consumo real e vínculo com unidade consumidora. A entidade deverá manter o padrão de arquitetura já existente no projeto e	Wendell Alves Nascimento , João Antonio Gomes Cestonaro	Completo	2 horas	2 horas

	possuir as validações necessárias na camada de serviço.				

Sprint Description

Tarefa: Sprint 16 - Correção e adição de atributos em Despesa

Objetivo

Corrigir e atualizar a entidade **Despesa** no sistema, ajustando seus atributos conforme a nova modelagem definida.

A alteração deve garantir que a entidade represente corretamente os dados de uma despesa vinculada a uma **Unidade Consumidora** e ao **Usuário** responsável pelo cadastro.

Descrição

Modificar a classe **Despesa** no backend, removendo campos que não serão mais utilizados diretamente e adicionando os novos atributos necessários para o controle de valores previstos, valores pagos, consumo previsto, consumo real e vínculo com unidade consumidora.

A entidade deverá manter o padrão de arquitetura já existente no projeto e possuir as validações necessárias na camada de serviço.

Critérios de Aceite

Atualização da Entidade Despesa

- Atualizar a classe `Despesa`
- Atualizar o mapeamento da entidade no banco de dados
- Criar a migration correspondente
- Atualizar a tabela `despesa` no banco de dados
- Garantir que a entidade siga o padrão utilizado no projeto

Atributos da Entidade

A classe `Despesa` deverá conter os seguintes campos:

- Id
 - Tipo serial/autoincremento
 - Chave primária da tabela
- NumeroDocumento
 - Tipo string
 - Representa o número do documento da despesa
- Codigo
 - Tipo string
 - Representa o código identificador da despesa
- DataEmissao
 - Deve representar a data de emissão da despesa
 - Analisar o melhor tipo a ser utilizado no projeto
- ValorPrevisto
 - Tipo decimal
 - Representa o valor previsto da despesa
- ValorPago
 - Tipo decimal
 - Representa o valor realmente pago
- ConsumoPrevisto
 - Tipo decimal
 - Representa o consumo previsto da despesa
- ConsumoReal
 - Tipo decimal
 - Representa o consumo real registrado
- DataVencimento

- Deve representar a data de vencimento da despesa
- Status
 - Deve utilizar o enum de status já existente no projeto
 - Representa a situação da despesa
- IdUsuario
 - Deve referenciar um usuário existente no banco
- IdUnidadeConsumidora
 - Deve referenciar uma unidade consumidora existente no banco

Campos que Devem ser Removidos ou Substituídos

- Remover o campo UC
- Remover o vínculo direto com TipoDespesa, caso ele passe a ser obtido pela UnidadeConsumidora
- Remover o vínculo direto com Orcamento, caso ele passe a ser obtido pela UnidadeConsumidora
- Remover o vínculo direto com Instituicao, caso ele passe a ser obtido pela UnidadeConsumidora
- Remover o vínculo direto com Fornecedor, caso ele passe a ser obtido pela UnidadeConsumidora

Validações de Campos Obrigatórios

- Validar que os campos obrigatórios não estão vazios:
 - NumeroDocumento
 - Codigo
 - DataEmissao
 - ValorPrevisto
 - ConsumoPrevisto
 - DataVencimento
 - Status
 - IdUsuario
 - IdUnidadeConsumidora

Validação de Valores

- ValorPrevisto não pode ser menor que zero
- ValorPago não pode ser menor que zero
- ConsumoPrevisto não pode ser menor que zero
- ConsumoReal não pode ser menor que zero
- ValorPago pode iniciar zerado caso a despesa ainda não tenha sido paga
- ConsumoReal pode iniciar zerado caso ainda não exista consumo realizado

Validação de Datas

- DataEmissao deve ser uma data válida
- DataVencimento deve ser uma data válida
- DataVencimento não deve ser anterior à DataEmissao

Validação de Status

- O campo Status deve aceitar apenas valores válidos do enum utilizado no projeto
- Uma nova despesa deve iniciar com o status padrão A PAGAR.
- Caso o status seja atualizado, validar se a alteração é permitida

Validação de Usuário

- IdUsuario deve existir no banco
- Validar integridade da chave estrangeira
- Garantir que o usuário esteja válido para uso

Validação de Unidade Consumidora

- IdUnidadeConsumidora deve existir no banco
- Validar integridade da chave estrangeira
- Garantir que a unidade consumidora esteja válida para uso
- Não permitir vínculo com unidade consumidora excluída por soft delete
- Não permitir cadastro de despesa para unidade consumidora inexistente ou inválida

Relacionamentos

Criar/ajustar o relacionamento da entidade Despesa com:

- Usuario
- UnidadeConsumidora
- Garantir que as chaves estrangeiras estejam configuradas corretamente
- Verificar se os nomes dos relacionamentos seguem o padrão do projeto

Regras de Negócio

- Não permitir cadastro de despesa com usuário inexistente
- Não permitir cadastro de despesa com unidade consumidora inexistente
- Não permitir cadastro de despesa com unidade consumidora excluída
- Garantir que apenas despesas válidas possam ser cadastradas, editadas e consultadas

Validação em Atualização

- Ao atualizar uma despesa:
 - Validar novamente todos os campos obrigatórios
 - Validar novamente o usuário informado
 - Validar novamente a unidade consumidora informada
 - Garantir que os novos dados estejam consistentes
 - Não permitir atualização vinculando a despesa a uma unidade consumidora excluída

Normalização e Boas Práticas

- Implementar validações na camada SERVICE
- Não depender apenas de DataAnnotations
- Validar dados antes de persistir no banco
- Retornar mensagens claras ao usuário
- Os erros retornados devem vir em uma lista com todos os erros encontrados de uma só vez
- Seguir o padrão de arquitetura já existente no projeto
- Verificar o projeto antes do envio

Endpoints e Resultado Esperado

Aqui estão os endpoints que esperamos ver na entrega da tarefa:

- `GET /api/despesas` - Retorna todas as despesas cadastradas.
- `GET /api/despesas/{id}` - Retorna uma despesa específica pelo ID.
- `GET /api/despesas/atrasadas` - Retorna todas as despesas atrasadas
- `GET /api/despesas/pagas` - Retorna todas as despesas pagas
- `GET /api/despesas/numero-documento/{numeroDocumento}` - Retorna despesas pelo número do documento.
- `GET /api/despesas/codigo/{codigo}` - Retorna despesas pelo código.
- `GET /api/despesas/unidade-consumidora/{idUnidadeConsumidora}` - Retorna despesas vinculadas a uma unidade consumidora.
- `GET /api/despesas/usuario/{idUsuario}` - Retorna despesas cadastradas por um usuário.
- `GET /api/despesas/status/{status}` - Retorna despesas filtradas pelo status.
- `POST /api/despesas` - Cadastrar novas despesas.
- `PUT /api/despesas/{id}` - Editar despesas existentes.
- `PATCH /api/despesas/{id}/status` - Alterar o status da despesa

Observações e Sugestões

- A entidade Despesa deve ser ajustada para depender da entidade UnidadeConsumidora.
- Os vínculos com instituição, tipo de despesa, secretaria, orçamento e fornecedor devem ser acessados por meio da unidade consumidora, caso essa seja a regra definida no projeto.
- Manter as validações centralizadas para facilitar manutenção futura.
- Verifiquem o projeto antes do envio.

Sprint Results

Sprint 16 — Atualização da entidade Despesa com Unidade Consumidora

Nesta sprint, foi realizada a atualização da modelagem da entidade Despesa no backend do Civitas, adequando-a ao novo vínculo obrigatório com Usuario e UnidadeConsumidora.

A implementação seguiu o contexto técnico definido na pasta .planning/codebase, mantendo a arquitetura existente do projeto e evitando alterações fora do escopo. A entidade UnidadeConsumidora foi criada de

forma mínima, sem controller público, apenas para dar suporte ao relacionamento necessário com Despesa.

Principais alterações realizadas:

- Atualização da entidade Despesa com os novos atributos:

- DataEmissao;
- ValorPrevisto;
- ValorPago;
- ConsumoPrevisto;
- ConsumoReal;
- IdUsuario;
- IdUnidadeConsumidora.

- Remoção dos vínculos diretos antigos de Despesa com:

- TipoDespesa;
- Orcamento;
- Instituicao;
- Fornecedor.

```

1  using Civitas.WebAPI.Objects.Enums;
2  using System.ComponentModel.DataAnnotations.Schema;
3
4  namespace Civitas.WebAPI.Objects.Models
5  {
6      /// <summary>
7      /// Entidade central que representa uma obrigação financeira, conta ou fatura a ser paga.
8      /// Mapeia a tabela 'despesa' do banco de dados.
9      /// </summary>
10     [Table("despesa")]
11     public class Despesa
12     {
13         /// <summary>
14         /// Identificador único da despesa (Chave Primária).
15         /// </summary>
16         [Column("id")]
17         5 referências
18         public int Id { get; set; }
19
20         /// <summary>
21         /// Número de identificação impresso no documento fiscal (NF, Fatura, Boletto).
22         /// </summary>
23         [Column("numerodocumento")]
24         4 referências
25         public string NumeroDocumento { get; set; }
26
27         /// <summary>
28         /// Código identificador da despesa.
29         /// </summary>
30         [Column("codigo")]
31         4 referências
32         public string Codigo { get; set; }
33
34         /// <summary>
35         /// Data em que o documento foi emitido.
36         /// </summary>
37         [Column("dataemissao")]
38         2 referências
39         public DateOnly DataEmissao { get; set; }
40
41         /// <summary>
42         /// Valor previsto para a despesa.
43         /// </summary>
44         [Column("valorprevisto")]
45         8 referências
46         public decimal ValorPrevisto { get; set; }
47
48         /// <summary>
49         /// Valor realmente pago.
50         /// </summary>
51         [Column("valorpago")]
52         2 referências
53         public decimal ValorPago { get; set; }
54
55         /// <summary>
56         /// Consumo previsto para esta despesa.
57         /// </summary>
58         [Column("consumoprevisto")]
59         2 referências
60         public decimal ConsumoPrevisto { get; set; }
61
62         /// <summary>
63         /// Consumo real registrado para esta despesa.
64         /// </summary>
65         [Column("consumoreal")]
66         2 referências
67         public decimal ConsumoReal { get; set; }
68
69         /// <summary>
70         /// Data limite para o pagamento sem incidência de juros ou multas.
71         /// </summary>
72     }
73 }

```

```

63     /// </summary>
64     [Column("datavencimento")]
65     2 referências
66     public DateOnly DataVencimento { get; set; }
67
68     /// <summary>
69     /// Status financeiro da despesa.
70     /// </summary>
71     [Column("status")]
72     5 referências
73     public Status Status { get; set; }
74
75     /// <summary>
76     /// Chave estrangeira do Usuário que cadastrou esta despesa.
77     /// </summary>
78     [Column("idusuario")]
79     4 referências
80     public int IdUsuario { get; set; }
81
82     /// <summary>
83     /// Objeto de navegação do Usuário.
84     /// </summary>
85     1 referência
86     public Usuario Usuario { get; set; }
87
88     /// <summary>
89     /// Chave estrangeira da unidade consumidora vinculada.
90     /// </summary>
91     [Column("idunidadeconsumidora")]
92     4 referências
93     public int IdUnidadeConsumidora { get; set; }
94
95     /// <summary>
96     /// Objeto de navegação da unidade consumidora.
97     /// </summary>
98     12 referências
99     public UnidadeConsumidora UnidadeConsumidora { get; set; }
100
101     0 referências
102     public Despesa()
103     {
104     }
105
106     3 referências
107     public Despesa(
108         int id,
109         string numeroDocumento,
110         string codigo,
111         DateOnly dataEmissao,
112         decimal valorPrevisto,
113         decimal valorPago,
114         decimal consumoPrevisto,
115         decimal consumoReal,
116         DateOnly dataVencimento,
117         Status status,
118         int idUsuario,
119         int idUnidadeConsumidora)
120     {
121         Id = id;
122         NumeroDocumento = numeroDocumento;
123         Codigo = codigo;
124         DataEmissao = dataEmissao;
125         ValorPrevisto = valorPrevisto;
126         ValorPago = valorPago;
127         ConsumoPrevisto = consumoPrevisto;
128         ConsumoReal = consumoReal;
129         DataVencimento = dataVencimento;
130         Status = status;
131         IdUsuario = idUsuario;
132         IdUnidadeConsumidora = idUnidadeConsumidora;
133     }
134 }

```

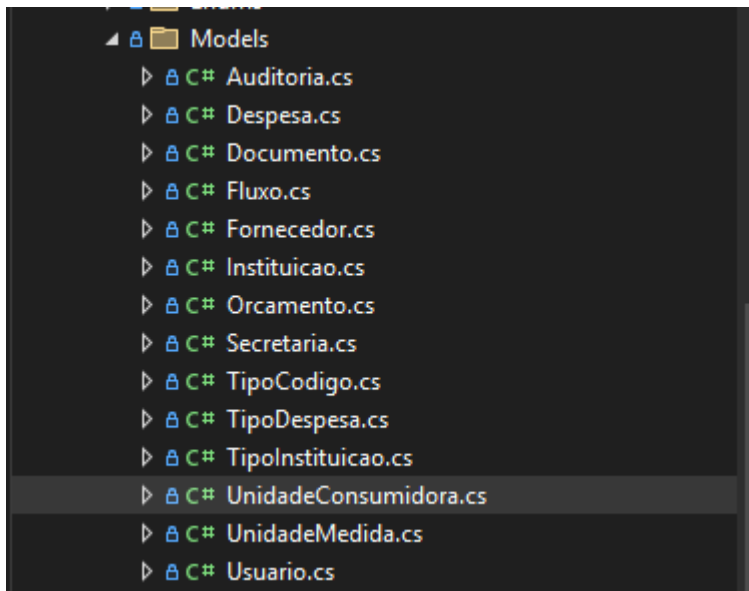
- Criação da entidade UnidadeConsumidora, contendo:

- Id;
- Codigo;
- Situacao;

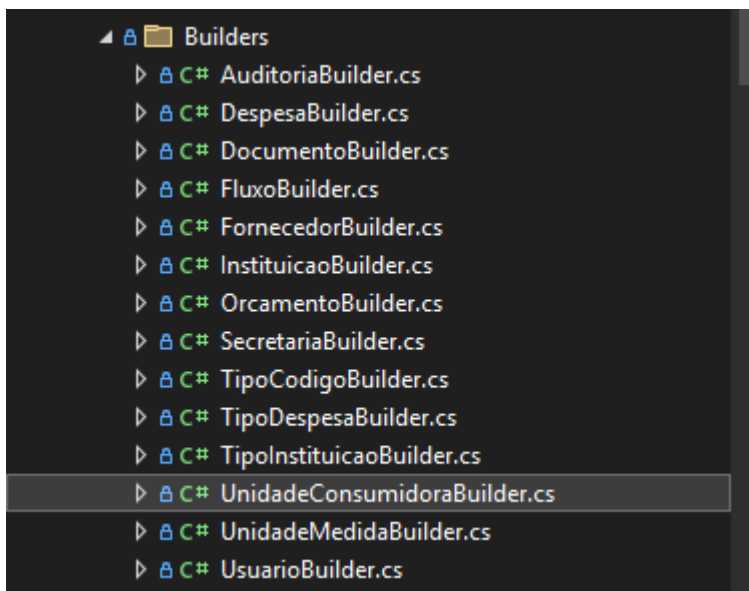
- IdTipoDespesa;
- IdOrcamento;
- IdInstituicao;
- IdFornecedor.

- Criação dos componentes necessários para UnidadeConsumidora:

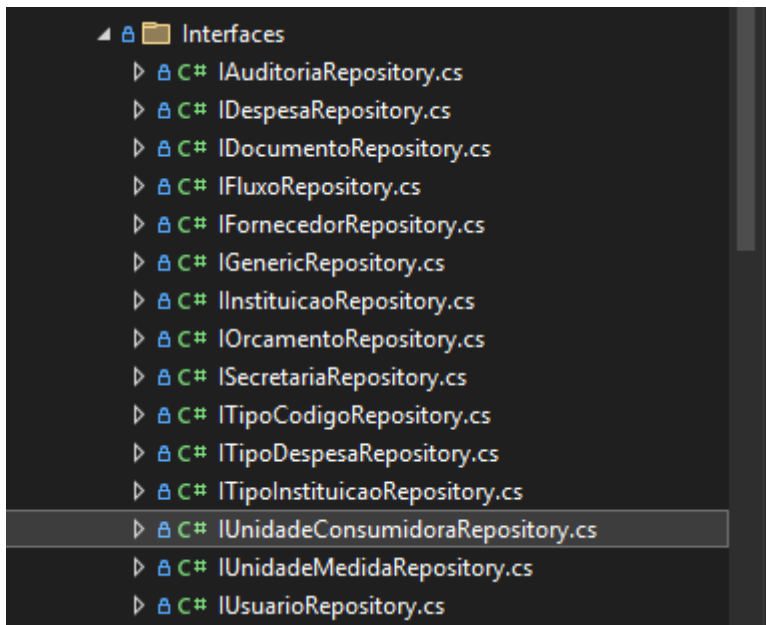
- model;



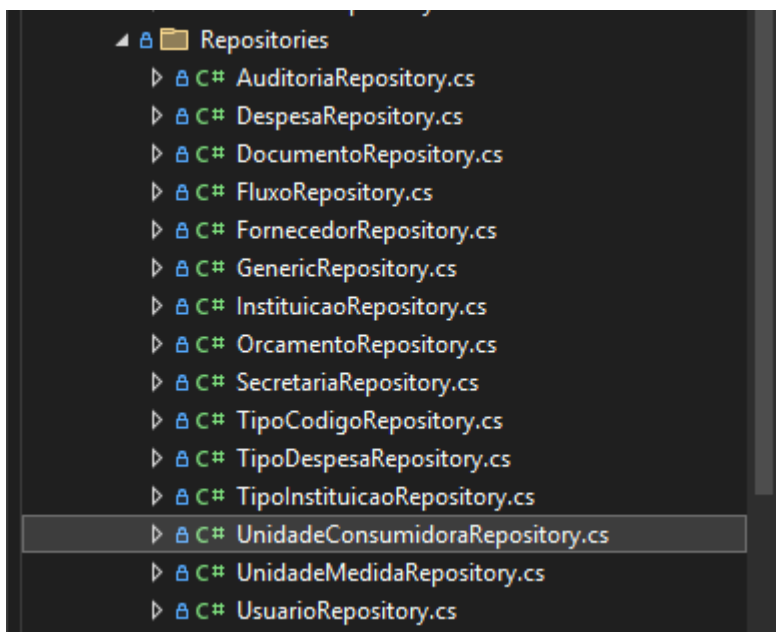
- builder;



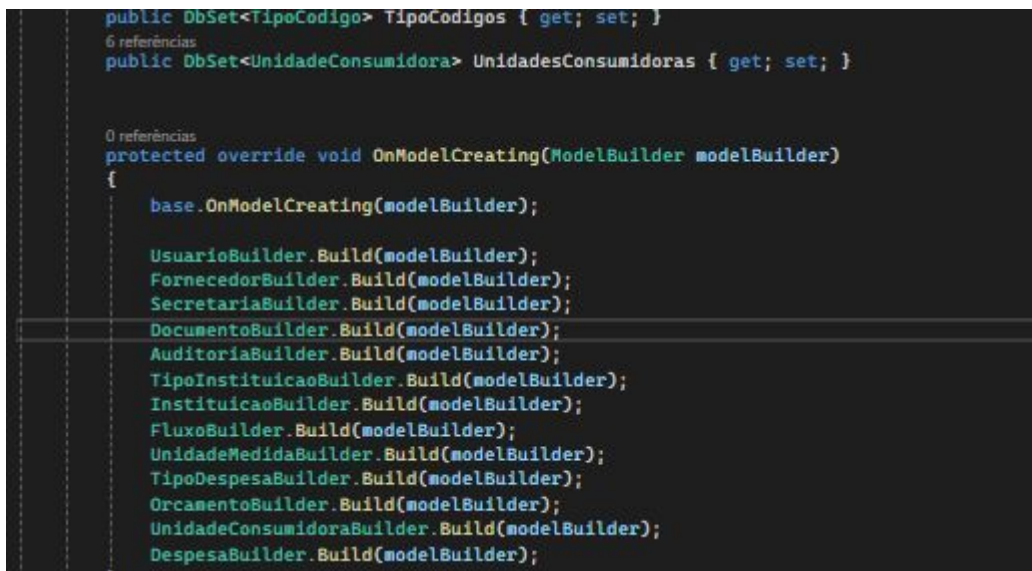
- interface de repository;



- repository;

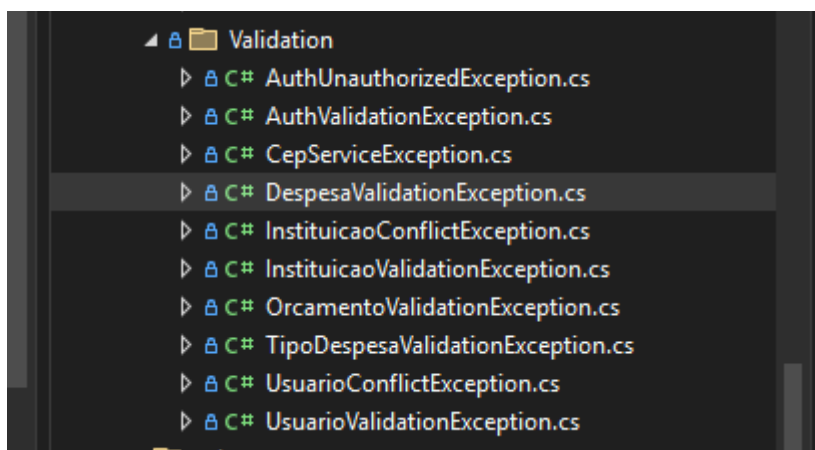


- DbSet;



- registro em injeção de dependência.

- Atualização dos arquivos relacionados a Despesa:
 - DespesaController;
 - DespesaService;
 - Despesa;
 - DespesaDTO;
 - DespesaBuilder;
 - DespesaRepository;
 - AutoMapper, quando aplicável.
- Centralização das validações de Despesa na camada de service, com acúmulo de erros em lista retornada no Response.Data.
- Criação da exceção DespesaValidationException para padronizar o retorno das validações.



- Aplicação das principais validações:
 - campos obrigatórios;
 - valores monetários e de consumo não negativos;
 - datas válidas;
 - vencimento não anterior à emissão;
 - status financeiro válido;
 - usuário existente e ativo;
 - unidade consumidora existente e ativa.
- Configuração do cadastro de novas despesas para iniciar automaticamente com Status.A_PAGAR.
- Ajuste da regra orçamentária existente para utilizar o vínculo com UnidadeConsumidora.IdOrcamento.

- Implementação dos endpoints solicitados para Despesa, incluindo:

- listagem geral;
- busca por ID;
- busca por despesas atrasadas;
- busca por despesas pagas;
- busca por número do documento;
- busca por código;
- busca por unidade consumidora;
- busca por usuário;
- busca por status;
- cadastro;
- atualização;
- alteração de status.

- Criação da migration DespesaUnidadeConsumidora, responsável por criar a tabela unidadeconsumidora e ajustar a tabela despesa conforme a nova modelagem.

- Atualização do snapshot do Entity Framework.

- Ajuste e criação de testes automatizados relacionados às validações e endpoints de Despesa.

Resumo técnico do diff:

- 26 arquivos alterados;
- 762 inserções;
- 700 deleções;
- 6 novos arquivos principais criados;
- 1 migration criada com arquivo Designer correspondente.

Arquivos principais alterados:

- Civitas.WebAPI/Controllers/DespesaController.cs
- Civitas.WebAPI/Services/Entities/DespesaService.cs

- Civitas.WebAPI/Objects/Models/Despesa.cs
- Civitas.WebAPI/Objects/Dtos/Entities/DespesaDTO.cs
- Civitas.WebAPI/Data/Builders/DespesaBuilder.cs
- Civitas.WebAPI/Data/Repositories/DespesaRepository.cs
- Civitas.WebAPI/Migrations/AppDbContextModelSnapshot.cs

Arquivos principais criados:

- Civitas.WebAPI/Objects/Models/UnidadeConsumidora.cs
- Civitas.WebAPI/Data/Builders/UnidadeConsumidoraBuilder.cs
- Civitas.WebAPI/Data/Interfaces/IUnidadeConsumidoraRepository.cs
- Civitas.WebAPI/Data/Repositories/UnidadeConsumidoraRepository.cs
- Civitas.WebAPI/Services/Validation/DespesaValidationException.cs
- Civitas.WebAPI.Tests/DespesaValidationEndpointsTests.cs

Migration criada:

- Civitas.WebAPI/Migrations/20260501005618_DespesaUnidadeConsumidora.cs
- Civitas.WebAPI/Migrations/20260501005618_DespesaUnidadeConsumidora.Designer.cs

Assinaturas



Documento assinado digitalmente
JOAO ANTONIO GOMES CESTONARO
Data: 02/05/2026 13:29:39-0300
Verifique em <https://validar.iti.gov.br>



Documento assinado digitalmente
WENDELL ALVES NASCIMENTO
Data: 02/05/2026 13:34:07-0300
Verifique em <https://validar.iti.gov.br>